

Table of contents

Chapter 9. Wiring your phone into your Linux machines	1
The two Wi-Fi tools (and why LocalSend is the one to start with)	1
LocalSend: your AirDrop replacement	2
KDE Connect: the optional companion	2
Do you actually need KDE Connect on an iPhone?	2
Install them	3
On the iPhone	3
On your Linux laptop	3
Open the firewall	3
Pair	4
Making it work <i>anywhere</i> : over Tailscale	4
Phone the headless Pi	4
The wired fallback: USB access with libimobiledevice	5
Where to start	6
Recap: and the series, complete	6

Chapter 9. Wiring your phone into your Linux machines

The payoff of this chapter: move files between your phone and your Linux laptop in one tap with LocalSend, the one app that’s all you need on an iPhone, reach your Pi the same way from anywhere over Tailscale, know when KDE Connect’s clipboard sync is worth adding (and when it isn’t), and have the one wired fallback (USB) for when Wi-Fi isn’t an option.

You’ve built a homelab your phone can reach. This part closes the loop in the *other* direction: getting your phone and your Linux machines to talk to each other for the everyday stuff, “send me that PDF,” “paste this URL onto my desktop,” “pull these photos off my phone.” On a Mac you’d reach for AirDrop and Handoff; on Linux you assemble the same conveniences from two small apps, plus one wired option you’ll rarely need.

Get one thing straight first: **these are three different tools solving three different problems.** Don’t think “which one wins”, think “which job am I doing.”

The two Wi-Fi tools (and why LocalSend is the one to start with)

Job	LocalSend	KDE Connect
Send / receive files	Yes	Yes
Share clipboard	No	Yes
Phone battery on desktop	No	Yes
Media remote control	No	Yes
Mirror notifications, SMS, run commands	No	Android only
Dead-simple, one-purpose app	Yes	No

! On an iPhone, LocalSend is the one to reach for

KDE Connect’s headline features, notification mirroring, replying to texts from your desktop, running commands on the phone, are **Android-only**: Apple’s sandbox forbids a third-party app from reading other apps’ notifications or sending SMS, so on iOS none of that works no matter what a generic comparison table claims. That leaves the iOS KDE Connect app doing just file transfer, clipboard sync, and battery status, in a clunkier, less polished package than LocalSend. For the job almost everyone actually has on a phone, moving files, **LocalSend is simpler, nicer-looking, more reliable in day-to-day use, and all you need**. Reach for KDE Connect only if you specifically want clipboard sync or battery on your desktop and don’t mind the extra setup; on an iPhone it’s genuinely optional, and plenty of people (me included) skip it. One more iOS quirk if you do run it: Apple’s background limits mean the KDE Connect app must be **open or recently used** to stay reachable, so it can’t sit dormant for days and still answer.

LocalSend: your AirDrop replacement

Think of LocalSend as a dedicated screwdriver: it does exactly one thing, extremely well. Open it, pick a nearby device, send the file. No pairing, no account, no integration, and it cheerfully moves a 5 GB video that you’d never want to push through a more “integrated” tool.

That’s its entire job, and it’s the right tool whenever the task is purely “get this file from here to there.” It also tends to *just work*: the connection finds the other device and the transfer goes through more consistently than KDE Connect’s does, and, just as important on an iPhone, **everything the app offers actually works**, nothing greyed out, Android-only, or quietly broken. That reliability is a big part of why it’s the one to lean on.

KDE Connect: the optional companion

KDE Connect is the multitool, and on Android it’s genuinely powerful. On an iPhone its reach is narrower: the one feature worth having is the **shared clipboard** (copy 192.168.1.50 on your phone, paste it on your desktop), with **battery status** on your desktop panel as a minor extra. The notification and SMS superpowers are Android-only, so on iOS that’s the whole story, and whether it’s worth a second, fiddlier app over just using LocalSend is a real question, answered next.

Do you actually need KDE Connect on an iPhone?

Honestly, probably not. LocalSend already covers the thing you do constantly, moving files, and it does it with a cleaner app and zero pairing. The only reason to add KDE Connect on an iPhone is if you want **clipboard sync** between phone and desktop often enough to justify a second, clunkier app. If that’s you, install both and let KDE Connect handle the clipboard while LocalSend handles files. If it isn’t, install LocalSend and stop there, you won’t miss anything. (On Android the math flips: KDE Connect’s notification and SMS features make it well worth running alongside LocalSend.)

Install them

On the iPhone

Install **LocalSend** from the App Store, that's the one you'll actually use. If you also want clipboard sync, grab **KDE Connect** too (the KDE Connect one is published by KDE e.V.); both are free.

On your Linux laptop

LocalSend ships a proper Debian package, so on Ubuntu/Debian the cleanest install is to grab the `.deb` from localsend.org/download and hand it to `apt`, which pulls in any dependencies and wires it into your normal updates:

```
# Ubuntu/Debian, run from wherever you saved the file (e.g. ~/Downloads).
# Pick x86-64 for an Intel/AMD laptop, arm-64 for an ARM one.
sudo apt install ./LocalSend-*-linux-x86-64.deb
```

The `./` matters: it tells `apt` to install that local file rather than look up a package by name. On Arch, install `localsend-bin` from the AUR. (A distro-agnostic AppImage is also on the [download page](#) if you'd rather not install anything.)

KDE Connect, from your distro's repos:

```
# Arch
sudo pacman -S kdeconnect

# Ubuntu/Debian
sudo apt install kdeconnect
```

Running KDE Connect on Hyprland (no Plasma, no GNOME)

KDE Connect does **not** require the full KDE Plasma desktop, but it does need its background daemon running and a way to interact with it. On a bare window manager like Hyprland:

- The daemon `kdeconnectd` is started on demand over D-Bus; launching the GUI (`kdeconnect-app`) or running `kdeconnect-cli -l` brings it up.
- For an at-a-glance tray/battery indicator, add a **Waybar** “custom” module that calls `kdeconnect-cli`, or just keep `kdeconnect-app` a keybind away.
- Skip **GSConnect**, it's a GNOME Shell extension and won't help on a bare Wayland setup like this. Plain `kdeconnect` is the right choice here.

Open the firewall

Both tools need their ports reachable on the LAN. If you run `ufw`:

```
sudo ufw allow 53317/udp      # LocalSend discovery + transfer
sudo ufw allow 53317/tcp
sudo ufw allow 1714:1764/udp  # KDE Connect
sudo ufw allow 1714:1764/tcp
```

If a transfer “sees” the device but hangs at 0%, a closed firewall port is the first thing to check.

Pair

- **LocalSend:** no pairing, open it on both devices, on the same Wi-Fi, and they appear in each other’s list. Send.
- **KDE Connect:** open the app on both, tap the laptop in the phone’s device list, and **accept the pairing request** on the desktop (`kdeconnect-app` or the CLI will prompt). Pairing is a one-time trust handshake.

Making it work *anywhere*: over Tailscale

Here’s the homelab tie-in. Out of the box, both tools find devices using **LAN multi-cast/broadcast discovery**, great at home, useless the moment your phone is on cellular, because broadcast packets don’t traverse a VPN. But your tailnet gives every device a stable `100.x.y.z` address, and both apps let you **add a device by IP manually**, which sidesteps discovery entirely:

- **LocalSend:** Settings → **add a favorite / manual device** → enter the target’s Tailscale IP (e.g. your laptop at `100.x.y.z`) with port 53317. Now you can send to your laptop from the airport.
- **KDE Connect:** the iOS app has an “**Add device by IP**” / refresh option, enter the laptop’s Tailscale IP. Pair once, and clipboard + file transfer work across the tailnet.

💡 Why this is the natural payoff of everything before it

This is the same trick that powered the whole series: discovery and “same network” assumptions break the instant you leave home, and Tailscale’s stable addresses are the fix. Just as you reached Audiobookshelf and `https://home.home` by tailnet IP, you reach your *laptop* by tailnet IP here. The phone Linux conveniences stop being a home-only luxury.

Phone the headless Pi

Keep one thing in mind here: LocalSend and KDE Connect are **desktop** tools. Your Pi from Chapters 1–5 runs a **headless** server OS (Ubuntu Server 26.04 LTS), no graphical desktop, so neither app is the natural fit there. For moving a file between your phone and the Pi, you already have better, GUI-free paths:

- **Drop it into a service you already run.** Files you want *on* the Pi (more audiobooks, say) can go straight into `~/audiobookshelf/media/Audiobooks`, `scp` them over the tailnet:

```
# from your laptop, over Tailscale, no LAN required
scp ~/Downloads/lesson.mp3
you@homelab:~/audiobookshelf/media/Audiobooks/
```

- **From the phone directly**, an iOS file-manager app that speaks SSH/SFTP (several exist) can connect to **homelab** over Tailscale and drop files onto the Pi without any desktop app in between.

In short: **laptop** **phone** is LocalSend/KDE Connect territory; **phone/laptop** **the headless Pi** stays SSH/scp territory, which you already have for free over Tailscale.

The wired fallback: USB access with libimobiledevice

Everything above is Wi-Fi. USB is a **completely separate technology stack**, and (honestly) most people rarely need it. Install it the day you actually hit one of these, not before:

- No Wi-Fi available and you need photos off the phone *now*.
- You're moving 100 GB of video and want the fastest possible transfer.
- The phone's networking is broken and you need emergency access.

When that day comes:

```
# Arch
sudo pacman -S libimobiledevice ifuse usbmuxd

# Ubuntu/Debian
sudo apt install libimobiledevice6 libimobiledevice-utils ifuse usbmuxd
```

Then plug in, trust the computer on the phone, and mount:

```
idevicepair pair                # tap "Trust" on the iPhone when prompted
mkdir -p ~/iphone
ifuse ~/iphone                  # mounts the phone's media (camera roll)
# ... copy your files ...
fusermount -u ~/iphone          # unmount when done
```

i What USB actually gives you (and what it doesn't)

usbmuxd is the daemon that talks to the phone over the USB cable; **libimobiledevice** is the library implementing Apple's protocols; **ifuse** mounts the result as a folder. Because of iOS's sandbox, **ifuse** exposes the **photos/ camera-roll area** (and, with extra flags, individual apps' document folders), *not* the whole iOS filesystem. So USB is excellent for bulk photo/video offload, but it isn't a "browse my entire iPhone like a USB stick" experience. No jailbreak, no full filesystem, that's an Apple limitation, not a missing package.

Where to start

On an iPhone, begin with one app:

1. **LocalSend**, the AirDrop replacement that handles the file job everyone actually has, and whose features all reliably work.

Add **KDE Connect** only if you find yourself wanting clipboard sync between phone and desktop; on Android, add it from the start, where its notification and SMS features earn their keep. Either way, set up the Tailscale manual-IP pairing once you want these to work away from home. Then, *if* months later you find yourself wanting to pull a whole camera roll off over a cable, install **libimobiledevice** + **ifuse** at that point, when you'll know exactly why you need them, instead of carrying tools you never touch.

For most people, Wi-Fi transfer (LocalSend / KDE Connect, extended over Tailscale) handles the overwhelming majority of phone → Linux moments. USB is a backup and a power-user tool, not a daily requirement.

Recap: and the series, complete

- **LocalSend** = the one to start with: dead-simple file send with a clean app whose features all actually work, install it on the iPhone and the laptop and you're set for the common case.
- **KDE Connect** = optional companion: handy for clipboard sync and battery, but on an iPhone it's limited (notifications/SMS are Android-only) and clunkier, so add it only if you want clipboard sync, it earns its place mainly on Android.
- **Tailscale extends both off-LAN**: add devices by their 100.x.y.z IP so transfer and clipboard work from anywhere, exactly as the rest of your homelab does.
- **The headless Pi** stays on SSH/scp over Tailscale, the right tool for a machine with no desktop.
- **USB (libimobiledevice + ifuse)** is the wired fallback for bulk photo offload or no-Wi-Fi emergencies, install it only when you actually need it.

That completes *Beginner Homelab on a Raspberry Pi*. You started by building a foundation (a hardened, always-on Pi on a private Tailscale network) and then gave it job after job: streaming your media, blocking ads on every device, clean `http://*.home` URLs behind a reverse proxy, a single `https://home.home` control panel, a deliberate answer for VPN privacy on the road, and now two-way file sharing with your phone. Every piece runs on hardware you own, over one private network, with nothing exposed to the public internet.